

LinearSkills Academy

Class 13 — Functions Basics
Zero to Advance

Part 1 — The Basics

1. What is a Function?

A function is a reusable block of code that performs a specific task. Instead of writing the same code again and again, you write it once and "call" it whenever needed.

```
def greet():  
    print("Hello, welcome!")  
  
greet()    # calling the function  
greet()    # can call it as many times as needed
```

2. def — Defining a Function

def is the keyword used to create a function.

```
def function_name():  
    # code goes here  
    pass
```

3. Parameters — Passing Data In

Parameters let you send values into a function so it can work with them.

```
def greet(name):  
    print("Hello,", name)  
  
greet("Ali")    # Hello, Ali  
greet("Sara")    # Hello, Sara
```

You can use more than one parameter:

```
def add(a, b):  
    print(a + b)  
  
add(5, 3)    # 8
```

4. return — Sending a Value Back

print() just displays something. return actually gives back a value that you can store and use later.

```
def add(a, b):  
    return a + b  
  
result = add(5, 3)  
print(result)    # 8
```

Common Mistake:

```
def add(a, b):  
    print(a + b)    # only shows the value, doesn't return it
```

```
result = add(5, 3)
print(result)    # None -- because nothing was returned
```

Part 2 — Going Further

5. Keyword Arguments

Normally values are matched to parameters by their position (a, then b). With keyword arguments, you name which parameter each value belongs to, so the order no longer matters.

```
def describe(name, age):
    print(name, "is", age, "years old")

describe(age=20, name="Ali")    # order doesn't matter here
# Output: Ali is 20 years old
```

6. Returning Multiple Values

A function can return more than one value at once. Python quietly packs them into a tuple.

```
def get_min_max(a, b, c):
    return min(a, b, c), max(a, b, c)

low, high = get_min_max(4, 9, 2)
print(low, high)    # 2 9
```

7. Functions Calling Other Functions

A function can use the result of another function — this is how large programs are built from small pieces.

```
def square(n):
    return n * n

def sum_of_squares(a, b):
    return square(a) + square(b)

print(sum_of_squares(2, 3))    # 4 + 9 = 13
```

8. Docstrings — Documenting a Function

A short description written right under def, inside triple quotes. It explains what the function does — good practice for real projects.

```
def add(a, b):
    """Returns the sum of two numbers."""
    return a + b
```

9. A Peek Ahead

Two things you will learn properly in the next class: giving a parameter a default value (so it becomes optional), and functions that accept an unknown number of arguments. For now, just know they exist.

```
def greet(name="friend"):    # default value -- covered next class
    print("Hello,", name)

greet()    # Hello, friend
greet("Ali")    # Hello, Ali
```

Part 3 — Python vs C++ Functions

10. Side-by-Side Comparison

Feature	Python	C++
Definition keyword	def	Return type (int, void, etc.)
Return type	Not declared	Must be declared
Block start/end	Colon : + indentation	Curly braces { }
Semicolons	Not needed	Required ;
Parameter types	Not declared (dynamic)	Must be declared (e.g. int a)

Example:

Python	C++
<pre>def add(a, b): return a + b</pre>	<pre>int add(int a, int b) { return a + b; }</pre>

Part 4 — Mini Demo & Practice

11. Mini Demo

```
def calculate_area(length, width):  
    area = length * width  
    return area
```

```
result = calculate_area(5, 3)  
print("Area:", result)
```

12. Practice Task

Write a function called `is_even` that takes one number as a parameter and returns `True` if it is even, or `False` if it is odd. Then call it with a few different numbers and print the results.